SAS Modbus Toolkit

Comprehensive User Guide

Version 1.0 | Industrial Network Diagnostic & Commissioning Tool

Table of Contents

1. Overview

The SAS Modbus Toolkit is a professional Windows desktop application developed by Southern Automation Solutions for industrial network diagnostics, device commissioning, and Modbus communication testing. It is designed for automation engineers, system integrators, and technicians who need a reliable, standalone diagnostic tool that works on any Windows PC without PLC software licenses.

1.1 Key Capabilities

- Modbus Master — Read and write registers on any Modbus TCP or RTU device
- Slave Simulator — Simulate a Modbus slave device for testing master applications
- Bus Scanner — Automatically discover devices on RS-485 buses and TCP networks
- Network Diagnostics — Measure communication quality with a 0–100 health score
- Data Calculator — Convert raw register values across 16+ data types
- Error Reference — Offline Modbus exception code and communication error guide

1.2 Supported Protocols

Protocol	Transport	Typical Use
Modbus TCP	Ethernet / Wi-Fi (port 502)	PLCs, drives, smart instruments with Ethernet ports
Modbus RTU	RS-485 serial (via USB adapter)	Legacy field devices, sensors, drives on RS-485 bus

1.3 System Requirements

- Windows 10 or Windows 11 (64-bit)
- Python 3.11+ (if running from source) — or use the pre-built .exe
- USB-to-RS485 adapter for RTU connections (FTDI or Prolific chip recommended)
- No PLC software, no licenses, no internet connection required

2. Quick Start Guide

2.1 Connecting as Modbus Master (TCP)

1. Launch the application and click Modbus Master in the left sidebar.
2. Select TCP as the connection mode in the Connection panel.
3. Enter the device IP Address, Port (default 502), and Slave ID (usually 1).
4. Set the Timeout to 1.0 second for LAN connections; 3.0+ for wireless or VPN.
5. Click Connect. The status badge turns green/blue when the connection succeeds.
6. The poll table will begin reading registers. Watch the Decoded Value column for live data.

ℹ *If the connection fails, verify the IP is reachable with a ping command first. Check the device documentation for its Unit ID — some devices use ID 0 or 255.*

2.2 Connecting as Modbus Master (RTU / RS-485)

7. Connect your USB-to-RS485 adapter and note the COM port in Device Manager.
8. Select RTU as the connection mode.
9. Choose the COM port from the dropdown and set the baud rate from your device manual.
10. Set Parity (N = None, E = Even, O = Odd), Stop Bits, and Slave ID to match the device.
11. Click Connect. If no response, try swapping the A and B wires on the RS-485 terminal.

ℹ *The most common RTU connection problem is baud rate mismatch. If the device manual is not available, try 9600 baud with No Parity first, then step up through 19200, 38400, and 115200.*

2.3 Reading Registers

The Poll Table in the Modbus Master view shows one or more rows, each reading a block of registers:

Column	Description
Addr (0-based)	Protocol address — 0 = first register in the bank
Reg #	Traditional 5-digit register number (auto-calculated from address and FC)
Count	How many consecutive registers to read in this row
Function Code	Which register bank to read (see Section 3 for full reference)
Tag Label	Your custom name for this register or group
Data Type	How to interpret the raw bits (see Section 4 for full reference)

Column	Description
Raw Value	The raw integer or hex value returned by the device
Decoded Value	The value after applying the selected data type interpretation
Status	OK / ERROR — colour coded

2.4 Writing Register Values

12. Scroll down to the Write Value panel under the poll table.
13. Select the function code: FC06 for a single register, FC16 for multiple registers.
14. Enter the Address (0-based), the value to write, and the Data Type.
15. Click Write. The status message confirms success or shows the error code.

ℹ *FC05 (Write Single Coil) uses 0xFF00 for ON and 0x0000 for OFF — not 1 and 0.*

3. Modbus Addressing Reference

3.1 Register Banks

Modbus defines four register banks, each accessed by a specific function code:

Register Bank	Read FC	Write FC	1-Based Range	0-Based Address	Data Width
Coils (R/W bits)	FC01	FC05 / FC15	00001 – 09999	0 – 9998	1 bit
Discrete Inputs (R/O bits)	FC02	—	10001 – 19999	0 – 9998	1 bit
Input Registers (R/O)	FC04	—	30001 – 39999	0 – 9998	16 bits
Holding Registers (R/W)	FC03	FC06 / FC16	40001 – 49999	0 – 9998	16 bits

3.2 0-Based vs 1-Based Addressing

Device manuals typically list registers in 1-based notation (e.g., 40001 for the first holding register). The Modbus wire protocol and this tool use 0-based addresses. The Reg # column automatically shows the traditional 1-based number for cross-reference.

Conversion Formula:

- Holding Register: Address = (Manual register number) - 40001
- Input Register: Address = (Manual register number) - 30001
- Discrete Input: Address = (Manual register number) - 10001
- Coil: Address = (Manual register number) - 00001

3.3 Quick Conversion Table

Manual Shows	Register Type	Enter this Address
40001	Holding Register	0
40002	Holding Register	1

Manual Shows	Register Type	Enter this Address
40100	Holding Register	99
40501	Holding Register	500
30001	Input Register	0
30050	Input Register	49
10001	Discrete Input	0
00001	Coil	0

3.4 Function Code Reference

FC	Name	Direction	Typical Use
FC01	Read Coils	Read	Digital outputs, control bits (1-bit R/W)
FC02	Read Discrete Inputs	Read	Push buttons, limit switches, sensor states (1-bit R/O)
FC03	Read Holding Registers	Read	Setpoints, parameters, most device data (16-bit R/W)
FC04	Read Input Registers	Read	Measured process variables, live sensor readings (16-bit R/O)
FC05	Write Single Coil	Write	Force one coil ON (0xFF00) or OFF (0x0000)
FC06	Write Single Register	Write	Write one 16-bit value to a holding register
FC15	Write Multiple Coils	Write	Write a block of coils in one transaction
FC16	Write Multiple Registers	Write	Write a block of registers — required for 32-bit values

4. Data Types & Decoding

4.1 Understanding Register Data

Every Modbus register holds 16 bits (2 bytes). The device manual specifies how to interpret those bits. Choosing the wrong data type will produce a meaningless value even though the communication itself is working correctly.

4.2 Data Type Reference Table

Data Type	Registers	Range / Format	Typical Use
UINT16	1	0 to 65,535	Unsigned integer — counters, raw values, percentages x10
INT16	1	-32,768 to +32,767	Signed integer — temperatures below zero
UINT32_AB	2	0 to 4,294,967,295	32-bit unsigned, high word first (most devices)
UINT32_BA	2	0 to 4,294,967,295	32-bit unsigned, low word first (some legacy devices)
INT32_AB	2	-2.1B to +2.1B	32-bit signed, high word first
INT32_BA	2	-2.1B to +2.1B	32-bit signed, low word first
FLOAT32_AB	2	IEEE 754 float	Most common float — Allen-Bradley, Siemens, most PLCs
FLOAT32_BA	2	IEEE 754 float (swapped)	Some older Modicon / GE devices
FLOAT32_ABCD	2	IEEE 754 float	Full big-endian byte order
FLOAT32_CDAB	2	IEEE 754 float	Full little-endian byte order
FLOAT64	4	Double precision	High-precision laboratory instruments
BCD16	1	0 – 9999	Older devices encoding decimal in hex digits (BCD)
ASCII	1+	2 chars per register	Device serial numbers, text strings

Data Type	Registers	Range / Format	Typical Use
BOOL_BIT0	1	0 or 1	Single flag bit packed in a holding register
HEX	1	0x0000 – 0xFFFF	Raw hex display — useful for debugging

4.3 Byte Order (Endianness)

When a value spans two registers (FLOAT32, INT32, UINT32), the order of the two 16-bit words matters. This is called byte order or endianness.

Example — the float value 1.0 is stored as 0x3F800000 in IEEE 754:

Data Type	Register[0] (first address)	Register[1] (next address)
FLOAT32_AB (big-endian, high word first)	0x3F80	0x0000
FLOAT32_BA (little-endian, low word first)	0x0000	0x3F80

ℹ *If your value looks like a garbage number (e.g., 4.97e-39), try the other byte order variant. Use the Data Calculator tab to test all 16 type interpretations on any raw register values simultaneously.*

4.4 Scaled Integer Values

Many devices store floating-point values as scaled integers to avoid the complexity of IEEE 754. For example, a temperature of 23.5 degC may be stored as 235 (multiply by 0.1). Always check the device manual for a scale factor.

- Divide by 10 for one decimal place (e.g., 235 -> 23.5)
- Divide by 100 for two decimal places (e.g., 2350 -> 23.50)
- Divide by 1000 for three decimal places (e.g., 23500 -> 23.500)

5. Modbus Master — Detailed Reference

5.1 Connection Panel

Field	Description
Mode	TCP for Ethernet/Wi-Fi devices; RTU for RS-485 serial devices
IP Address	TCP: the device's IPv4 address (e.g., 192.168.1.10)
Port	TCP: usually 502; some devices use 503, 1502, or a custom port
Slave ID	The Modbus unit identifier (1–247). Most devices default to 1
COM Port	RTU: the Windows COM port of your USB-to-RS485 adapter
Baud Rate	RTU: must match device exactly — check device manual (common: 9600, 19200, 38400)
Parity	RTU: N=None, E=Even, O=Odd — must match device setting
Stop Bits	RTU: usually 1; use 2 only when device documentation requires it
Timeout	Seconds to wait for a response before marking a timeout error

5.2 Poll Table

Each row in the poll table defines an independent read request that executes on every poll cycle. You can add up to 20 rows. Rows run sequentially within each poll interval.

To read a 32-bit float value that spans two registers: set Count = 2 and Data Type = FLOAT32_AB (or _BA). The two registers will be decoded together as one floating-point number.

5.3 Transaction Log

Every read and write operation is recorded in the transaction log with a timestamp, slave ID, function code, address, value, and response time. The log stores the last 500 transactions and can be exported to CSV for analysis or customer documentation.

6. Slave Simulator

6.1 Overview

The Slave Simulator turns your PC into a Modbus slave device. This is useful for testing PLC programs, HMI configurations, or master applications without needing physical hardware. It supports both Modbus TCP (any Ethernet master can connect) and Modbus RTU (via RS-485 adapter — useful for loop-back testing with two adapters).

6.2 Register Banks

All four Modbus register banks are available with 1000 registers each:

- Holding Registers (HR) — 16-bit read/write (FC03/FC06/FC16)
- Input Registers (IR) — 16-bit read-only from the master perspective (FC04)
- Coils — 1-bit read/write (FC01/FC05/FC15)
- Discrete Inputs (DI) — 1-bit read-only from the master perspective (FC02)

ℹ *All values start at 0 when the server starts. Live edits take effect immediately while the server is running.*

6.3 Simulation Mode

Enabling Simulation Mode automatically varies several registers to simulate a live device:

Register	Behavior	Useful For
HR 0 (address 0)	Sine wave 0–1000 (period ~12 seconds)	Testing chart trends and graphing
HR 1 (address 1)	Ramp 0–65535 over 30 seconds, then reset	Testing counter roll-over handling
HR 2 (address 2)	Incrementing counter (wraps at 65535)	Verifying continuous poll connectivity
IR 0 (address 0)	Random walk 200–800 (simulates temp x10)	Testing sensor value handling
Coil 0 (address 0)	Toggles ON/OFF every 2 seconds	Testing coil state handling

7. Bus Scanner

7.1 RTU Bus Scan

Scans a range of slave IDs on an RS-485 serial bus. Each ID is probed with an FC03 read request. The scanner also tests FC01, FC02, and FC04 to determine which register banks each device supports. Results include slave ID, response time, and a list of supported function codes.

> ℹ *RTU scanning requires exclusive access to the serial port. Close any other applications (PLC software, HMI, other serial tools) before scanning.*

7.2 TCP Single IP Scan

Scans a range of unit IDs (slave IDs) on a single IP address. This is most useful when a Modbus TCP-to-RTU gateway is present — the gateway forwards each unit ID to a different device on the serial bus behind it.

7.3 TCP Network Range Scan

Scans a range of IP addresses on the same subnet for Modbus TCP devices. Each IP is tested for port 502 connectivity and a valid Modbus response. Useful for auditing an industrial network or discovering undocumented devices.

7.4 Device Identity (FC43)

When a device supports the FC43 MEI Read Device Identification request, the scanner attempts to retrieve the vendor name, product name, and firmware version automatically. Many devices do not support FC43 — this is normal and does not affect other results.

8. Network Diagnostics

8.1 Health Score

The health score is a 0–100 composite metric calculated from three components:

Component	Weight	Measurement	Threshold
Error Rate	50%	Percentage of failed transactions	0% = full score; 5%+ = zero score
Response Time	30%	Average response time in ms	< 10ms = full; 500ms = zero score
Jitter (Variation)	20%	Standard deviation of response times	< 5ms = full; 200ms = zero score

Score	Rating	Typical Cause
90 – 100	Excellent	Clean wired Ethernet or well-terminated RS-485
70 – 89	Good	Minor noise or occasional retries — acceptable for most applications
50 – 69	Fair	Noticeable errors or slow responses — investigate before deployment
0 – 49	Poor	Significant communication problems — not suitable for production use

8.2 Diagnostic Report

Clicking Analyze Now generates a written report with findings categorized by severity:

- CRITICAL — communication is unreliable; immediate action required
- WARNING — degraded performance; investigation recommended
- CAUTION — minor issues; monitor over time
- INFO — baseline observations; no action needed

The report can be exported as a text file for customer documentation or field reports.

9. Data Calculator

9.1 Register Value Converter

Enter one or two raw register values (as decimal integers or hex, e.g., 0x3F80) and instantly see the value interpreted as all 16 supported data types. This is the fastest way to determine the correct data type when reading an unknown device.

9.2 Additional Reference Tables

The Data Calculator tab also contains the following offline reference materials:

- Byte Order Reference — visual diagrams showing AB vs BA word arrangement
- Function Code Reference — quick-reference table of all Modbus function codes
- Address Reference — 0-based to 1-based conversion for all four register banks
- RTU Timing Reference — minimum silence gap requirements at common baud rates
- Modbus Exception Codes — all 9 protocol exception codes with causes and fixes
- Common Communication Errors — 20+ real-world error scenarios with troubleshooting steps

10. Troubleshooting

10.1 No Response — Diagnostic Checklist

TCP connections:

16. Ping the device IP from a command prompt to confirm basic network reachability.
17. Verify port (default 502). Try 503, 1502, or check device documentation for a custom port.
18. Check that Windows Firewall is not blocking port 502 on inbound or outbound.
19. Verify the Slave/Unit ID. TCP devices commonly use ID 0 or 1. Some gateways use 255.
20. Confirm the device is configured for Modbus TCP server mode (not just Modbus RTU mode).

RTU connections:

21. Verify the COM port number in Device Manager > Ports (COM & LPT).
22. Confirm no other application is using the same COM port.
23. Try baud rates in order: 9600, 19200, 38400, 115200.
24. Try swapping the A and B signal wires on the RS-485 terminal — polarity is the #1 RTU problem.
25. Verify parity (N/E/O) and stop bits from the device manual.
26. Check for missing termination resistors (120 ohm at each physical end of the cable).

10.2 Exception Code Reference

Code	Name	Common Cause	Fix
0x01	Illegal Function	Device does not support that FC	Use Bus Scanner to find supported FCs
0x02	Illegal Data Address	Register does not exist at that address	Check device manual; verify 0-based vs 1-based
0x03	Illegal Data Value	Count=0, or value out of allowed range	FC05: use 0xFF00 or 0x0000; max 125 regs per request
0x04	Server Device Failure	Internal device hardware/firmware fault	Power cycle device; contact manufacturer
0x06	Server Device Busy	Device processing long operation	Retry after 1–2 sec; reduce poll rate
0x0A	Gateway Path Unavailable	Gateway serial port busy or misconfigured	Check gateway configuration and slave ID routing

Code	Name	Common Cause	Fix
0x0B	Gateway No Response	Serial device not responding to gateway	Check serial wiring; verify slave ID; use RTU Scanner

10.3 CRC Errors and Noise (RTU)

- Route RS-485 cable away from VFD output cables and power wiring.
- Ensure cable shield is connected at the master end only to prevent ground loops.
- Use optically isolated RS-485 adapters in high-EMI environments (near VFDs, welders).
- Add or verify termination resistors (120 ohm) at both physical ends of the cable.
- Use shielded twisted pair cable — Belden 9841 or equivalent.
- Reduce baud rate if cable is long (see RTU Timing Reference in Data Calculator).

11. RS-485 Wiring Guide

11.1 Wiring Checklist

Item	Requirement	Notes
Termination	120 ohm across A-B at EACH physical end	Never in the middle; most common omission
Bias Resistors	560 ohm pull-up on B(+) and pull-down on A(-)	On ONE device only — usually the master
Polarity	A(-) to A(-), B(+) to B(+) across all devices	Swap A/B if no devices respond
Shield Ground	Connect shield at master end ONLY	Grounding at both ends causes ground loop noise
Cable Type	Shielded twisted pair (STP)	Belden 9841 or equivalent; not regular wire
Cable Routing	Separate cable tray from power wiring	Keep away from VFD output cables especially
Max Devices	32 unit loads per segment	Use RS-485 repeater to extend beyond 32 nodes

11.2 Cable Length vs Baud Rate

Baud Rate	Max Cable Length (approx.)	Notes
9,600 bps	~1200 m (3900 ft)	Suitable for plant-wide RS-485 runs
19,200 bps	~600 m (1950 ft)	Most common baud for industrial devices
38,400 bps	~300 m (975 ft)	Good balance of speed and distance
57,600 bps	~200 m (650 ft)	Use only for shorter runs
115,200 bps	~100 m (325 ft)	Panel-level or short-distance only

11.3 USB-to-RS485 Adapter Recommendations

Chip	Notes	Best For
FTDI FT232R / FT232H	Best driver support; reliable timing	All applications — preferred choice
Prolific PL2303	Widely available; works well	General use
CH340 / CH341	Budget option; some timing issues at 115200+	Short-distance, lower baud rates
Isolated (FTDI-based)	Optical isolation; more expensive	Near VFDs, welders, or noisy environments

12. Contact & Support

For questions, custom integrations, or on-site support:

Company	Southern Automation Solutions
Owner	Mark Patera
Email	Mark@SASControls.com
Phone	229-563-2897
Address	111 Hemlock St. Ste A, Valdosta, GA 31601
Website	SASControls.com